Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «КиСБ»
Тема: «Блочные шифры и классические режимы шифрования»

Выполнил:
Студент 4-го курса
Группы АС-66
Езепчук А.С.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: программно реализовать симметричный блочный шифр AES-128, режимы ECB и CFB, а также дополнение PKCS#7 без сторонних криптографических библиотек.

Вариант 3

Задание:

1. Реализовать зашифрование и расшифрование одного 128-битного блока AES-128 и генерацию раундовых ключей.

2. Реализовать ECB с дополнением PKCS#7 и CFB-128 для сообщений произвольной длины.

3. Продемонстрировать, что при расшифровании CFB используется только функция зашифрования блока E_K.

4. Проверить граничные длины сообщений и сравнить режимы на данных с повторяющимися блоками.

Теоретическая часть

AES-128 — SP-сеть с блоком и ключом по 16 байт и 10 раундами. Состояние представляется матрицей 4 × 4 байта с заполнением по столбцам: s[r,c] = input[4c+r]. Начальное преобразование AddRoundKey выполняет XOR состояния с ключом K0.

Раунды 1–9 включают SubBytes, ShiftRows, MixColumns и AddRoundKey. В десятом раунде MixColumns отсутствует. SubBytes заменяет байты по S-box; ShiftRows циклически сдвигает строку r влево на r позиций. MixColumns умножает столбец на матрицу над $GF(2^8)$ со строками (02 03 01 01), (01 02 03 01), (01 01 02 03), (03 01 01 02).

Умножение на 2: xtime(b) = ((b << 1) XOR q) AND 0xFF, где q = 0x1B при установленном старшем бите, иначе q = 0. Умножение на 3 равно xtime(b) XOR b. Поле задаётся многочленом $x^8+x^4+x^3+x+1$.

Ключ расширяется до 44 слов по 4 байта. Если i кратно 4, Wi = Wi−4 XOR SubWord(RotWord(Wi−1)) XOR Rcon[i/4]; иначе Wi = Wi−4 XOR Wi−1. RotWord сдвигает байты на одну позицию, SubWord применяет S-box. Получаются 11 раундовых ключей.

Расшифрование начинает с AddRoundKey с ключом K10, затем применяет InvShiftRows, InvSubBytes, AddRoundKey и InvMixColumns. В последнем обратном раунде InvMixColumns пропускается. Обратная S-box строится из прямой таблицы.

Режимы шифрования и структура программы

ECB: Ci = E_K(Pi); Pi = D_K(Ci). Каждый блок обрабатывается независимо. Одинаковые блоки открытого текста дают одинаковые блоки шифротекста, поэтому режим раскрывает повторяющиеся структуры данных.

PKCS#7: k = 16 − (L mod 16). В конец добавляются k байт со значением k. При L = 16 добавляется целый блок из 16 байт 0x10. При снятии проверяются кратность длины, диапазон k от 1 до 16 и совпадение всех последних k байт.

CFB-128: C0 = IV; Ci = Pi XOR E_K(Ci−1); Pi = Ci XOR E_K(Ci−1). В обоих направлениях используется E_K, а обратная связь содержит шифротекст. Для последнего сегмента длины m < 16 берутся первые m байт гаммы. Паддинг не требуется.

Корректность расшифрования CFB следует из свойства XOR: (Pi XOR E_K(Ci−1)) XOR E_K(Ci−1) = Pi. Использование D_K не требуется.

Весь исполняемый код находится в одном файле lab3.py. Внутри файла выделены логические разделы: блочный шифр AES128, функции режимов и PKCS#7, класс тестов Lab3Tests, демонстрация run_demo и файловый интерфейс file_main. Режимы используют интерфейс AES128, не реализуя его раундовую логику повторно.

Функции encrypt_block(pt_block, key) и decrypt_block(ct_block, key) соответствуют заданию. Длины ключа и блока строго проверяются. Расширение ключа выполняется один раз на экземпляр AES128.

Запуск программы

```
python lab3.py test
python lab3.py demo
python lab3.py encrypt cfb input.txt encrypted.bin \
  --key 000102030405060708090a0b0c0d0e0f
python lab3.py decrypt cfb encrypted.bin restored.txt \
  --key 000102030405060708090a0b0c0d0e0f
```

Для ECB следует заменить cfb на ecb. Требуется Python 3.10 или новее; установка библиотек не нужна. Выходной файл должен отсутствовать. CFB-файл содержит случайный 16-байтовый IV, затем шифротекст. Учебная демонстрация использует фиксированные ключ и IV для воспроизводимости.

Функции CFB обрабатывают сообщение целиком. Последний неполный сегмент допускается, однако повторный вызов с исходным IV не продолжает предыдущий поток.

Результаты тестирования

После объединения кода выполнено: python lab3.py test. Результат: Ran 10 tests, OK. Проверены контрольные векторы FIPS 197 и NIST SP 800-38A, UTF-8, бинарные данные, ошибочные длины и паддинг.

Контроль AES-128: ключ 000102030405060708090a0b0c0d0e0f; блок 00112233445566778899aabbccddeeff; полученный шифротекст 69c4e0d86a7b0430d8cdb78070b4c55a совпал с ожидаемым. Обратное преобразование восстановило блок.

Для ECB и CFB-128 совпали все четыре блока контрольных векторов NIST. Для CFB дополнительно проверены префиксы длиной 1, 15, 17, 31 и 63 байта. Тесты восстановления выполнены для длин 0, 1, 2, 15, 16, 17, 31, 32, 33, 127 и 256 байт.

Сообщение	Длина	PKCS#7	ECB	CFB
Hello	5	11 × 0B	16	5
0123456789ABCDEF	16	16 × 10	32	16
0123456789ABCDEFG	17	15 × 0F	32	17
Пустое	0	16 × 10	16	0

Длины приведены в байтах, значения PKCS#7 — в hex. Во всех случаях исходные данные восстановлены. В таблице не учтён 16-байтовый IV файлового формата CFB.

Особенность варианта 3: в test_cfb_uses_encrypt_only функция AES128.decrypt_block заменена исключением. Расшифрование 64 байт CFB прошло успешно; encrypt_block вызвана 4 раза. Таким образом проверено фактическое отсутствие вызовов D_K.

Ошибочный PKCS#7, ключи и блоки неправильной длины, а также IV не из 16 байт отклоняются. Через файловый интерфейс дополнительно проверено точное восстановление 262-байтового бинарного файла в обоих режимах.

Опыт с восемью одинаковыми блоками b"REPEATED BLOCK!!": среди первых восьми блоков ECB получен один уникальный шифроблок, CFB — восемь. Блок паддинга ECB при подсчёте не учитывался.

Сравнение режимов на изображении

Для наглядности зашифрован массив пикселей узора размером 128 × 96. Заголовок PGM оставлен открытым. Для изображения ECB отброшен только дополнительный блок PKCS#7; файловый интерфейс сохраняет шифротекст полностью.



Рисунок 1. Исходное изображение, ECB и CFB-128.

В ECB повторения открытых блоков приводят к повторению шифроблоков: видны вертикальные полосы. CFB скрывает исходный узор в этом опыте благодаря меняющейся обратной связи. Визуальное сравнение иллюстрирует недостаток ECB и не заменяет криптографический анализ.

Вывод

В ходе лабораторной работы реализованы AES-128, ECB с PKCS#7 и CFB-128 без сторонних криптографических библиотек. Весь код собран в одном файле с логически разделёнными частями. Корректность подтверждена контрольными векторами и десятью автоматическими тестами. Проверено, что расшифрование CFB использует только функцию E_K. На повторяющихся блоках и изображении показано раскрытие структуры данных в режиме ECB.

Реализация учебная. ECB и CFB не обеспечивают аутентификацию: неверный ключ или изменение шифротекста не всегда обнаруживаются. Для каждого нового CFB-файла создаётся случайный IV. Проверка PKCS#7 не заявляется как защита от padding oracle или timing-атак.

Источники

1. Задание лабораторной работы №3, вариант 3.
2. FIPS PUB 197. Advanced Encryption Standard.
3. NIST SP 800-38A. Режимы ECB и CFB, приложения F.1 и F.3.
4. RFC 5652, раздел 6.3. Дополнение блоков.

Листинг программы

Полный исходный код файла lab3.py приведён ниже.

```
"""Лабораторная работа №3, вариант 3: AES-128, ECB и CFB-128.
Python 3.10+. Только стандартная библиотека.
Запуск: python lab3.py demo | test | encrypt/decrypt ...
Блочный шифр и режимы логически разделены внутри одного файла.
"""
# 1. БЛОЧНЫЙ ШИФР AES-128
BLOCK_SIZE = 16
SBOX = bytes.fromhex('''
637c777bf26b6fc53001672bfed7ab76 ca82c97dfa5947f0add4a2af9ca472c0
b7fd9326363ff7cc34a5e5f171d83115 04c723c31896059a071280e2eb27b275
09832c1a1b6e5aa0523bd6b329e32f84 53d100ed20fcb15b6acbbe394a4c58cf
d0efaafb434d338545f9027f503c9fa8 51a3408f929d38f5bcb6da2110fff3d2
cd0c13ec5f974417c4a77e3d645d1973 60814fdc222a908846eeb814de5e0bdb
e0323a0a4906245cc2d3ac629195e479 e7c8376d8dd54ea96c56f4ea657aae08
ba78252e1ca6b4c6e8dd741f4bbd8b8a 703eb5664803f60e613557b986c11d9e
e1f8981169d98e949b1e87e9ce5528df 8ca1890dbfe6426841992d0fb054bb16
''')
INV_SBOX = bytes(SBOX.index(i) for i in range(256))
def _validate(value, size, name):
    if not isinstance(value, bytes):
        raise TypeError(f'{name}: ожидается bytes')
    if len(value) != size:
        raise ValueError(f'{name}: требуется {size} байт')
def xtime(x):
    return ((x << 1) ^ (0x1B if x & 0x80 else 0)) & 0xFF
def _mul(a, b):
    result = 0
    while b:
        if b & 1:
            result ^= a
        a = xtime(a)
        b >>= 1
    return result
def expand_key(key):
    """11 раундовых ключей по 16 байт (44 слова)."""
    _validate(key, 16, 'Ключ AES-128')
    expanded = list(key)
    rc = 1
    while len(expanded) < 176:
        t = expanded[-4:]
        if len(expanded) % 16 == 0:
            t = [SBOX[x] for x in t[1:] + t[:1]]
            t[0] ^= rc
            rc = xtime(rc)
        for x in t:
            expanded.append(expanded[-16] ^ x)
    return tuple(bytes(expanded[i:i+16]) for i in range(0, 176, 16))
def _add_key(state, key):
    return [a ^ b for a, b in zip(state, key)]
def _shift_rows(s, inverse=False):
    # Индекс строки r, столбца c: 4*c+r (порядок AES по столбцам).
    direction = -1 if inverse else 1
    return [s[4*((c + direction*r) % 4)+r]
            for c in range(4) for r in range(4)]
def _mix_columns(s, inverse=False):
    matrix = ((14,11,13,9),(9,14,11,13),(13,9,14,11),(11,13,9,14)) if inverse else (
        (2,3,1,1),(1,2,3,1),(1,1,2,3),(3,1,1,2))
    out = []
    for start in range(0,16,4):
        col = s[start:start+4]
        for row in matrix:
            v = 0
            for a,b in zip(row,col):
                v ^= _mul(b,a)
            out.append(v)
    return out
class AES128:
    """Расширение ключа выполняется один раз на экземпляр."""
    def __init__(self, key):
        self.round_keys = expand_key(key)
    def encrypt_block(self, pt_block):
        _validate(pt_block, 16, 'Блок')
        s = _add_key(pt_block, self.round_keys[0])
        for r in range(1,11):
            s = _shift_rows([SBOX[x] for x in s])
            if r != 10:
                s = _mix_columns(s)
            s = _add_key(s, self.round_keys[r])
        return bytes(s)
    def decrypt_block(self, ct_block):
        _validate(ct_block, 16, 'Блок')
```

```
        s = _add_key(ct_block, self.round_keys[10])
        for r in range(9,-1,-1):
            s = [INV_SBOX[x] for x in _shift_rows(s, inverse=True)]
            s = _add_key(s, self.round_keys[r])
            if r != 0:
                s = _mix_columns(s, inverse=True)
        return bytes(s)
def encrypt_block(pt_block, key):
    return AES128(key).encrypt_block(pt_block)
def decrypt_block(ct_block, key):
    return AES128(key).decrypt_block(ct_block)
# 2. РЕЖИМЫ И PKCS#7
def _bytes(data):
    if not isinstance(data, bytes):
        raise TypeError('Данные должны иметь тип bytes')
def pad_pkcs7(data):
    _bytes(data)
    k = BLOCK_SIZE - len(data) % BLOCK_SIZE
    return data + bytes([k]) * k
def unpad_pkcs7(data):
    _bytes(data)
    if not data or len(data) % BLOCK_SIZE:
        raise ValueError('Некорректный PKCS#7')
    k = data[-1]
    if not 1 <= k <= BLOCK_SIZE or data[-k:] != bytes([k]) * k:
        raise ValueError('Некорректный PKCS#7')
    return data[:-k]
def _xor(a, b):
    return bytes(x ^ y for x,y in zip(a,b))
def encrypt_ecb(data, key):
    _bytes(data)
    cipher = AES128(key)
    padded = pad_pkcs7(data)
    return b''.join(cipher.encrypt_block(padded[i:i+16])
                    for i in range(0,len(padded),16))
def decrypt_ecb(data, key):
    _bytes(data)
    cipher = AES128(key)
    if not data or len(data) % BLOCK_SIZE:
        raise ValueError('ECB: шифротекст должен содержать целые блоки')
    padded = b''.join(cipher.decrypt_block(data[i:i+16])
                      for i in range(0,len(data),16))
    return unpad_pkcs7(padded)
def _cfb(data, key, iv, decrypt):
    _bytes(data)
    _bytes(iv)
    if len(iv) != BLOCK_SIZE:
        raise ValueError('CFB: IV должен содержать 16 байт')
    cipher = AES128(key)
    feedback = iv
    result = bytearray()
    for i in range(0,len(data),BLOCK_SIZE):
        block = data[i:i+BLOCK_SIZE]
        # В обеих ветвях используется исключительно E_K.
        gamma = cipher.encrypt_block(feedback)
        transformed = _xor(block, gamma[:len(block)])
        result.extend(transformed)
        feedback = block if decrypt else transformed
    return bytes(result)
def encrypt_cfb(data, key, iv):
    return _cfb(data, key, iv, decrypt=False)
def decrypt_cfb(data, key, iv):
    return _cfb(data, key, iv, decrypt=True)
# 3. АВТОМАТИЧЕСКИЕ ТЕСТЫ
import random
import unittest
from unittest.mock import patch
H = bytes.fromhex
KEY = H('2b7e151628aed2a6abf7158809cf4f3c')
IV = H('000102030405060708090a0b0c0d0e0f')
PT = H('6bc1bee22e409f96e93d7e117393172a'
       'ae2d8a571e03ac9c9eb76fac45af8e51'
       '30c81c46a35ce411e5fbc1191a0a52ef'
       'f69f2445df4f9b17ad2b417be66c3710')
CFB = H('3b3fd92eb72dad20333449f8e83cfb4a'
        'c8a64537a0b3a93fcde3cdad9f1ce58b'
        '26751f67a3cbb140b1808cf187a4f4df'
        'c04b05357c5d1c0eeac4c66f9ff7f2e6')
ECB = H('3ad77bb40d7a3660a89ecaf32466ef97'
        'f5d3d58503b9699de785895a96fdbaaf'
        '43b1cd7f598ece23881b00e3ed030688'
        '7b0c785e27e8ad3f8223207104725dd4')
class Lab3Tests(unittest.TestCase):
    def test_fips197(self):
        key = H('000102030405060708090a0b0c0d0e0f')
```

```
        pt = H('00112233445566778899aabbccddeeff')
        ct = H('69c4e0d86a7b0430d8cdb78070b4c55a')
        self.assertEqual(encrypt_block(pt,key),ct)
        self.assertEqual(decrypt_block(ct,key),pt)
        self.assertEqual(expand_key(key)[10].hex(), '13111d7fe3944a17f307a78b4d2b30c5')
    def test_nist_ecb(self):
        cipher = AES128(KEY)
        for i in range(0,64,16):
            self.assertEqual(cipher.encrypt_block(PT[i:i+16]),ECB[i:i+16])
            self.assertEqual(cipher.decrypt_block(ECB[i:i+16]),PT[i:i+16])
        # API ECB добавляет обязательный блок PKCS#7.
        self.assertEqual(encrypt_ecb(PT,KEY)[:64],ECB)
        self.assertEqual(decrypt_ecb(encrypt_ecb(PT,KEY),KEY),PT)
    def test_nist_cfb128(self):
        self.assertEqual(encrypt_cfb(PT,KEY,IV),CFB)
        self.assertEqual(decrypt_cfb(CFB,KEY,IV),PT)
        for n in (1,15,17,31,63):
            self.assertEqual(encrypt_cfb(PT[:n],KEY,IV),CFB[:n])
            self.assertEqual(decrypt_cfb(CFB[:n],KEY,IV),PT[:n])
    def test_cfb_uses_encrypt_only(self):
        with patch.object(AES128,'decrypt_block',side_effect=AssertionError('D_K вызван')):
            with patch.object(AES128,'encrypt_block',autospec=True,
                              side_effect=AES128.encrypt_block) as encrypt:
                self.assertEqual(decrypt_cfb(CFB,KEY,IV),PT)
                self.assertEqual(encrypt.call_count,4)
    def test_lengths_and_binary(self):
        rng = random.Random(3)
        for n in (0,1,2,15,16,17,31,32,33,127,256):
            with self.subTest(n=n):
                data = bytes(rng.randrange(256) for _ in range(n))
                e = encrypt_ecb(data,KEY)
                c = encrypt_cfb(data,KEY,IV)
                self.assertEqual(len(e),(n//16+1)*16)
                self.assertEqual(len(c),n)
                self.assertEqual(decrypt_ecb(e,KEY),data)
                self.assertEqual(decrypt_cfb(c,KEY,IV),data)
    def test_utf8(self):
        data = 'Лабораторная работа №3 🔐'.encode('utf-8')
        self.assertEqual(decrypt_ecb(encrypt_ecb(data,KEY),KEY),data)
        self.assertEqual(decrypt_cfb(encrypt_cfb(data,KEY,IV),KEY,IV),data)
    def test_padding_all_sizes(self):
        for n in range(33):
            data = b'A'*n
            padded = pad_pkcs7(data)
            k = 16-n%16
            self.assertEqual(padded[-k:],bytes([k])*k)
            self.assertEqual(unpad_pkcs7(padded),data)
    def test_bad_padding(self):
        for data in (b'',b'a',b'A'*15+b'\0',b'A'*15+b'\x11',b'A'*14+b'\x03\x02'):
            with self.subTest(data=data), self.assertRaises(ValueError):
                unpad_pkcs7(data)
    def test_validation(self):
        for n in (0,15,17,24,32):
            with self.assertRaises(ValueError): AES128(b'k'*n)
        cipher = AES128(KEY)
        for n in (0,15,17):
            for fn in (cipher.encrypt_block,cipher.decrypt_block):
                with self.assertRaises(ValueError): fn(b'x'*n)
        for n in (0,15,17):
            for fn in (encrypt_cfb,decrypt_cfb):
                with self.assertRaises(ValueError): fn(b'',KEY,b'i'*n)
        for data in (b'',b'x',b'x'*17):
            with self.assertRaises(ValueError): decrypt_ecb(data,KEY)
        with self.assertRaises(TypeError): encrypt_ecb('text',KEY)
        with self.assertRaises(TypeError): AES128('k'*16)
    def test_patterns(self):
        data = b'REPEATED BLOCK!!'*8 # 16 байт
        e,c = encrypt_ecb(data,KEY),encrypt_cfb(data,KEY,IV)
        blocks = len(data)//16
        self.assertEqual(len({e[i*16:(i+1)*16] for i in range(blocks)}),1)
        self.assertEqual(len({c[i*16:(i+1)*16] for i in range(blocks)}),blocks)
# 4. ДЕМОНСТРАЦИЯ
from pathlib import Path
KEY = bytes.fromhex('2b7e151628aed2a6abf7158809cf4f3c')
IV = bytes(range(16))
OUT = Path(__file__).resolve().parent / 'results'
def run_demo():
    OUT.mkdir(exist_ok=True)
    lines = ['Лабораторная работа №3. Вариант 3',
             'AES-128; ECB + PKCS#7; CFB-128 без паддинга',
             'Ключ (учебный): '+KEY.hex(), 'IV (учебный): '+IV.hex(), '']
    for data in (b'Hello',b'0123456789ABCDEF',b'0123456789ABCDEFG',
                 'Привет, мир!'.encode('utf-8'),b''):
        e,c = encrypt_ecb(data,KEY),encrypt_cfb(data,KEY,IV)
        lines += [f'Исходные данные: {data!r}; длина {len(data)} байт',
```

```
                'PKCS#7: '+pad_pkcs7(data).hex(),
                f'ECB ({len(e)} байт): '+e.hex(),
                f'CFB ({len(c)} байт): '+c.hex(),
                f'Восстановление ECB: {decrypt_ecb(e,KEY)==data}',
                f'Восстановление CFB: {decrypt_cfb(c,KEY,IV)==data}', '']
    data = b'REPEATED BLOCK!!'*8
    e,c = encrypt_ecb(data,KEY),encrypt_cfb(data,KEY,IV)
    lines += ['Сравнение восьми одинаковых блоков (без блока паддинга ECB)',
              '№ | исходный блок | ECB | CFB']
    for i in range(8):
        lines.append(f'{i+1} | {data[i*16:i*16+16].hex()} | '
                     f'{e[i*16:i*16+16].hex()} | {c[i*16:i*16+16].hex()}')
    lines += ['Уникальных блоков ECB: 1; CFB: 8', '',
              'Проверка CFB без D_K: см. test_cfb_uses_encrypt_only в lab3.py.',
              'D_K заменяется исключением; восстановление проходит, E_K вызывается 4 раза.']
    # Наглядные PGM: заголовок открыт, шифруется только массив пикселей.
    w,h = 128,96
    pixels = bytes(220 if ((x//16+y//16)%2) else 35 for y in range(h) for x in range(w))
    for name,body in [('original',pixels),('ecb',encrypt_ecb(pixels,KEY)[:len(pixels)]),
                      ('cfb',encrypt_cfb(pixels,KEY,IV))]:
        (OUT/f'{name}.pgm').write_bytes(f'P5\n{w} {h}\n255\n'.encode()+body)
    output = '\n'.join(lines)+'\n'
    (OUT/'demo_output.txt').write_text(output,encoding='utf-8')
    print(output)
# 5. ФАЙЛОВЫЙ ИНТЕРФЕЙС
import argparse
from pathlib import Path
from secrets import token_bytes
def file_main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('action',choices=['encrypt','decrypt'])
    p.add_argument('mode',choices=['ecb','cfb'])
    p.add_argument('input',type=Path)
    p.add_argument('output',type=Path)
    p.add_argument('--key',required=True,help='32 шестнадцатеричных символа (16 байт)')
    args = p.parse_args()
    try:
        key = bytes.fromhex(args.key)
        if len(key) != 16:
            raise ValueError('Требуется ключ AES-128 длиной 16 байт')
        if args.input.resolve() == args.output.resolve():
            raise ValueError('Входной и выходной файлы должны различаться')
        if args.output.exists():
            raise ValueError('Выходной файл уже существует')
        data = args.input.read_bytes()
        if args.mode == 'ecb':
            result = (encrypt_ecb if args.action=='encrypt' else decrypt_ecb)(data,key)
        elif args.action == 'encrypt':
            iv = token_bytes(16)
            result = iv + encrypt_cfb(data,key,iv)
        else:
            if len(data) < 16:
                raise ValueError('В CFB-файле отсутствует полный IV')
            result = decrypt_cfb(data[16:],key,data[:16])
        with args.output.open('xb') as target:
            target.write(result)
    except (ValueError,TypeError,OSError) as exc:
        p.exit(1,f'Ошибка: {exc}\n')
    print(f'Готово: {args.output}; {len(result)} байт')
if __name__ == '__main__':
    import sys
    if len(sys.argv) == 1 or sys.argv[1] == 'demo':
        run_demo()
    elif sys.argv[1] == 'test':
        suite = unittest.defaultTestLoader.loadTestsFromTestCase(Lab3Tests)
        result = unittest.TextTestRunner(verbosity=2).run(suite)
        sys.exit(0 if result.wasSuccessful() else 1)
    else:
        file_main()
```